

CHIP 2026 Hackathon — Tim J5 (ITB)

Laporan Fase 3

Eksplorasi Multi-lane $L = 1 / 2 / 4 / 8$, Eksperimen Optimasi, dan L Terpilih

Status: **PARTIAL** — titik operasi terpilih C2-K2-K1 pada $L = 8$; timing belum terpenuhi; Approval belum dicentang

Branch: phase3-multilane · HEAD saat laporan dibuat: 0ca6890 · diperbarui 2026-09-30

Ditulis untuk: Tim J5. Sumber kebenaran tetap berkas di repo (docs/results/result_phase3.md, docs/evidence/phase03-multilane/, docs/evidence/quartus/, docs/decisions/); dokumen ini rangkuman bacaan. Versi ini menggantikan laporan Fase 3 tanggal 2026-09-29: kesimpulan formal di versi lama sudah diralat (Bagian 9) dan L terpilih berubah dari $L=4$ menjadi $L=8$ (Bagian 10).

1. Tujuan dan Lingkup

Fase 3 mengukur *trade-off* sumber daya terhadap performa dari beberapa butterfly paralel per siklus (*lane*) di atas memori ter-bank Fase 2, untuk $L = 1, 2, 4, 8$ (konfigurasi **C2**), lalu memilih satu titik operasi. Kriteria pemilihan L ditetapkan lewat ADR *sebelum sweep* Quartus diukur.

Setelah *sweep* C2 selesai, tim meminta dua eksperimen optimasi tambahan (K2 dan K1) untuk melihat apakah $L = 8$ bisa masuk anggaran ALM tanpa mengurangi 8 butterfly per siklus. Eksperimen itu dijalankan sebagai konfigurasi terpisah; RTL dan bukti C2 tidak diubah.

2. Kriteria Pemilihan L (ADR 0004)

- **Primer:** jumlah siklus paling sedikit, di antara L yang pemakaian ALM-nya tidak melebihi 25% perangkat target (5CSEBA6U23I7, 41.910 ALM), yaitu **10.478 ALM** (perhitungan tim).
- **Sekunder** (informasional): evaluasi AT setelah ada *clock* yang memenuhi timing. Belum bisa dijalankan karena belum ada konfigurasi yang memenuhi timing.

3. Arsitektur Multi-lane

Lane ke- l memproses indeks butterfly $p = l \cdot (128/L) + t$. Inti C0/C1 memakai satu penghitung indeks zeta sekuensial, yang tidak bisa dipakai bila beberapa lane berada di blok berbeda pada siklus yang sama. C2 memakai rumus tertutup per lane:

NTT (maju): $k = 2^{\text{lapis}} + \text{blok}$

INTT (balik): $k = 127 - (\text{jumlah blok di lapis-lapis sebelumnya}) - \text{blok}$

Rumus ini dicocokkan secara menyeluruh dengan aturan sekuensial C0: 0 selisih dari 127 pasangan (lapis, blok) pada kedua arah, dan cakupan indeks per lane lengkap tanpa duplikat untuk semua L

(lane_schedule_verification_2026-09-29.txt). Memori poly_mem_multiport menyediakan $2 \cdot L$ port logis di atas pemetaan bank Fase 2.

4. Hasil C2 (MEASURED)

Besaran	$L=1$	$L=2$	$L=4$	$L=8$
ALM	6.018 (14%)	5.728 (14%)	7.629 (18%)	11.446 (27%), di atas anggaran
Register	3.100	3.098	3.102	3.100
Blok M10K	0/553	0/553	0/553	0/553
Blok DSP	3	5	9	17
Fmax, Slow 100°C	14,76 MHz	13,54 MHz	11,60 MHz	7,62 MHz
Slack setup terburuk @ 20 ns	-47,733 ns	-54,644 ns	-66,690 ns	-111,219 ns
Siklus NTT / INTT (simulasi)	897 / 1153	449 / 705	225 / 481	113 / 369

Sumber: docs/evidence/quartus/C2-L{1,2,4,8}-20260929.md dan cocotb_regression_2026-09-29.txt (16/16 di Icarus dan Verilator; $L=1$ identik dengan C0/C1). Timing tidak terpenuhi di semua L dan memburuk seiring L naik; *pipelining* adalah lingkup Fase 4.

Dengan ADR 0004 pada keluarga C2, $L = 8$ tersingkir oleh anggaran ALM (lebih 968 ALM), sehingga hasilnya **$L = 4$** . Hasil ini dipertahankan sebagai pembanding dasar.

5. Audit ALM $L = 8$

Rincian per-entitas dari laporan *fitter* (l8_opt_entity_breakdown_2026-09-30.txt) untuk C2-L8:

Kelompok	ALM	Catatan
Memori (poly_mem_multiport + bank_map_rom)	6.660,1 (58,2%)	3.072 flip-flop penyimpanan, mux, crossbar 16 port ke 8 bank

Kelompok	ALM	Catatan
Aritmetika butterfly	3.569,9 (31,2%)	tiap lane punya dua pengali modular (maju dan balik), masing-masing dengan pembagi untuk operator %
Alamat, kontrol, zeta	1.026,8 (9,0%)	FSM, pembangkit alamat dan indeks zeta per lane, twiddle_rom
Pengali penskalaan INTT	187,3 (1,6%)	hanya aktif pada tahap penskalaan

Quartus tidak membagi dua pengali per lane itu, padahal mode maju/balik tetap selama satu proses. Tanpa menyentuh butterfly atau memori, logika yang tersisa hanya sekitar 1.214 ALM, sehingga penghematan 968 ALM tidak mungkin dicapai dari sana saja.

6. Eksperimen K2 dan K1

- **K2** — penghitung sub-siklus t_q dilebarkan sesuai kebutuhan tiap L (4 bit pada L=8, bukan 8 bit). Masih dalam lingkup Fase 3.
- **K1** — satu pengali modular per butterfly yang dipakai bergantian untuk mode maju dan balik (`butterfly_shared.sv`), di atas K2: konfigurasi **C2-K2-K1**. Persamaannya sama dengan butterfly asli dan `modmul_reduce.sv` tidak diubah. K1 mengubah datapath butterfly, jadi berada di luar lingkup tertulis Fase 3; tim menyetujuinya sebagai eksperimen tambahan.

ALM (MEASURED)	L=1	L=2	L=4	L=8
C2	6.018	5.728	7.629	11.446
C2-K2	6.389	5.788	7.600	11.232
C2-K2-K1	5.566	5.374	6.775	9.754
C2-K2-K1 dalam anggaran 10.478?	ya	ya	ya	ya (724 di bawah)
DSP, C2 → C2-K2-K1	3 → 2	5 → 3	9 → 5	17 → 9
Fmax C2-K2-K1, Slow 100°C	14,33 MHz	12,63 MHz	10,89 MHz	7,68 MHz
Slack setup terburuk C2-K2-K1	-49,804 ns	-59,148 ns	-71,868 ns	-110,494 ns
Siklus NTT / INTT	897 / 1153	449 / 705	225 / 481	113 / 369

Sumber: docs/evidence/quartus/C2-L{1,2,4,8}-K2-20260930.md dan C2-K2-K1-L{1,2,4,8}-20260930.md, perangkat, *constraint* dan *seed* sama dengan C2. Pada L=8, 16 pengali per lane (3.068,7 ALM) menjadi 8 (1.563,5 ALM); mux operand menambah 48,3 ALM.

Catatan: K2 bukan perbaikan yang seragam (−214 ALM pada L=8, −29 pada L=4, +60 pada L=2, +371 pada L=1). Kenaikan pada L=1 seluruhnya berada di blok memori dengan jumlah ALUT dan register yang identik, yaitu logika yang sama dikemas berbeda oleh *fitter*. Jadi angka ALM dapat bergeser beberapa ratus ALM antar-kompilasi tanpa perubahan logika, sedangkan margin C2-K2-K1 L=8 terhadap anggaran adalah 724 ALM. *Seed sweep* belum dijalankan.

7. Verifikasi K1

Pengujian	Hasil
Uji unit <code>butterfly_shared</code> terhadap model emas (kasus tepi + 1.000 acak per mode)	2/2 di Verilator dan Icarus
Regresi inti C2-K2-K1, L = 1/2/4/8 (bit-exact, bolak-balik, siklus konstan, <code>bank_overflow_o</code>)	16/16 di Verilator dan Icarus; siklus identik dengan C2
Ekuivalensi <code>butterfly</code> vs <code>butterfly_shared</code> , formal dengan pengali diabstraksi	PASS; dua kontrol negatif gagal sebagaimana mestinya
Ekuivalensi yang sama, simulasi menyeluruh pada RTL asli	73.785.560.578 kombinasi (2×3329^3), 0 selisih
Bukti formal ekuivalensi dengan aritmetika penuh	tidak selesai dalam 30 menit; dicatat sebagai catatan, bukan hasil

Bukti formal memakai satu asumsi yang dinyatakan eksplisit: `modmul_reduce` adalah fungsi murni dari masukannya. Simulasi menyeluruh tidak memakai asumsi. Sumber: `k1_cocotb_regression_2026-09-30.txt`, `k1_equiv_abstraction_2026-09-30.txt`, `k1_exhaustive_equivalence_2026-09-30.txt`.

8. Verifikasi Formal: Hasil

Seluruh bukti dijalankan dengan satu skrip, `formal/run_formal_slang.py`: 19 dari 19 hasil sesuai harapan (`formal_rerun_2026-09-30.md`).

Kelompok	Hasil
C2, C2-K2, C2-K2-K1 pada $L = 1/2/4/8$ (12 bukti)	12/12 PASS, k-induction
Kontrol negatif: salinan <code>bank_map_rom</code> yang sengaja dirusak (4)	gagal di basecase atau di langkah induksi, sesuai rancangan
Bukti Fase 1 dan Fase 2 dijalankan ulang dengan alur yang sama (3)	3/3 tetap PASS

Yang terbukti: `bank_overflow_o` tidak pernah aktif (tidak lebih dari dua akses ke satu bank per siklus), jabat tangan `busy_o` $\rightarrow$ `done_o`, dan rentang `t_q` serta `layer_q`. **Yang tidak dibuktikan secara formal:** kebenaran bit-exact NTT/INTT, integritas data memori, dan bahwa proses pasti selesai; untuk itu buktinya adalah regresi simulasi dan ekuivalensi butterfly di Bagian 7.

9. Ralat atas Laporan Sebelumnya

Laporan 2026-09-29 menyebut $L = 2/4/8$ "belum tuntas" dan menduga penyebabnya cara pembaca Yosys membaca tabel case 256-entri. Status saat itu UNKNOWN (basecase lulus, induksi gagal), bukan FAIL. Lokasinya benar, mekanismenya keliru.

Akar masalah yang ditemukan dari analisis *trace*: Yosys mengubah tabel `bank_map_rom` dan `twiddle_rom` menjadi sel memori. Pada langkah induksi, isi memori dianggap *state* bebas, sehingga *solver* memakai ROM yang isinya bukan tabel sebenarnya dan "menemukan" pelanggaran. Itu *state* yang tidak dapat dicapai: artefak perangkat uji formal, bukan masalah RTL.

Perbaikan hanya pada perangkat uji: ROM dipetakan kembali menjadi logika konstan (`memory_map -rom-only`), *frontend* diganti ke `read_slang` karena pembaca bawaan membiarkan hasil `add_mod/sub_mod` tanpa penggerak, dan asumsi reset ditulis ulang secara ekuivalen. **RTL tidak diubah.**

10. L Terpilih (ADR 0005)

Keputusan tim (Faza Dzil, Tim J5, 2026-09-30): konfigurasi teroptimasi **C2-K2-K1** diadopsi dan **$L = 8$** dipilih — 9.754 ALM (di bawah anggaran 10.478), NTT 113 / INTT 369 siklus, 8 butterfly per siklus. Fase 4 berangkat dari C2-K2-K1 pada $L = 8$.

Ini deviasi yang disadari dan dicatat dari lingkup tertulis Fase 3, karena K1 mengubah datapath butterfly. Metode reduksi modular dan parameter FIPS 203 tidak berubah. ADR 0004 tidak disunting; ADR 0005 menerapkannya pada keluarga C2-K2-K1. Hasil C2 ($L = 4$) tetap disimpan sebagai pembandingan.

11. Status dan Batasan

Fase 3 **PARTIAL**. CRG-9 (timing) gagal untuk semua konfigurasi pada *clock* sementara 20 ns. CRG-8 (formal) kini lulus untuk properti kontrol dan kapasitas bank. Kotak Approval pada `docs/results/result_phase3.md` **belum dicentang**.

- Timing belum terpenuhi dan belum ada ADR *clock* target; seluruh F_{max} bersifat relatif.
- Margin 724 ALM lebih besar dari, tetapi tidak jauh di atas, variasi pengemasan *fitter* terbesar yang teramati (sekitar 370 ALM).
- 0 blok M10K terpakai (warisan Fase 2, baca asinkron).
- K1 sedikit menurunkan F_{max} pada $L = 1/2/4$.
- Belum ada validasi di papan DE10-Nano; semua angka berasal dari Quartus dan simulasi.

12. Cara Reproduksi

```
. scripts/env.sh
python3 scripts/gen_lane_schedule.py
python3 tb/ntt/run_ntt_c2_tests.py verilator      # C2; tambahkan k2 atau k1 untuk varian
python3 tb/ntt/run_k1_unit_tests.py verilator
python3 formal/run_formal_slang.py               # 19 hasil formal
tb/ntt/k1_exhaustive/run_k1_exhaustive.sh         # sekitar 20 menit pada 8 core
# Quartus: revision C2-L<n>, C2-L<n>-K2, C2-K2-K1-L<n> di quartus/phase03_multilane_c2/
```